

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 02 — PLC PROGRAMMING LANGUAGES

Practical Lab 02

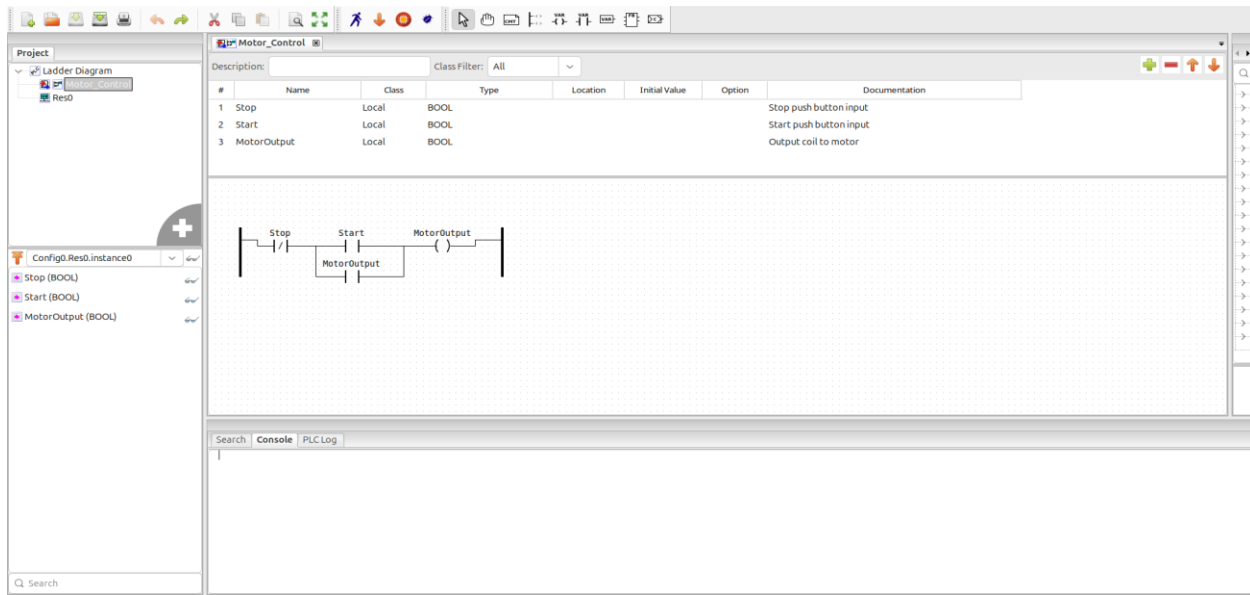
PLC Programming Languages and Motor Control Logic

Item	Details
Course	ACICP301 – ICS & SCADA Systems (Practical Laboratories)
Lab	Practical Lab 02: PLC Programming Languages and Motor Control Logic
Platform	ICDFA Offline ICS/PLC Virtual Laboratory – OpenPLC VM
Due	02 Oct 2026, 11:59 PM WAT
Practical Pass	50%
Student Name	Nathaniel Temitope Ogunjobi
Registration Number	C11/26/ACIS/17372
Date	22 September 2026

Authorization & Safety: this practical was performed only within the ICDFA-provided PLC virtual machine, kept on the instructor-specified isolated/host-only network and not bridged to any production ICS/SCADA system. A clean starting snapshot was preserved before beginning. Evidence below is labelled Evidence 1–7 per the portal's submission requirement, covering all steps in the official lab brief.

Evidence 1: OpenPLC Environment and Ladder Diagram Motor-Control Program

The ICDFA PLC VM was started from its clean snapshot and signed into using the provided credentials. OpenPLC Editor was launched from the desktop menu, and via File > Open, the Ladder Diagram project was opened from the Downloads/PLC Programming Language Examples directory. Under the Project tab, Res0 was expanded and instance0 (Motor_Control) opened via its Edit symbol, with the pane expanded to full screen.



Evidence 1 screenshot: OpenPLC Editor with the Motor_Control Ladder Diagram program fully expanded, program name (Motor_Control) clearly visible

Evidence 2: Start/Stop and Latching Logic Explanation

The Motor_Control program is a basic motor start/stop control for a system with a start button, a stop button, and a motor, implemented as a standard latching (seal-in) circuit in Ladder Diagram.

Rails and rungs: the two vertical PowerRails represent the beginning (left) and end (right) of the logic, named after the original relay-logic wires from the main power supply. The horizontal rung between them is read top to bottom, and the conditions on the left of the rung must be true for the output on the right to activate.

Start contact (NO): a normally open contact. Its gap is open by default (false state, no signal passes); pressing the Start button closes the gap (true state), allowing the signal through. Since the button press is temporary, this alone would not hold the motor on.

MotorOutput latch contact (NO): a second normally open contact referencing the MotorOutput coil itself, wired in parallel with Start. Once MotorOutput energizes, this contact closes and maintains (“latches” or “seals in”) the circuit independently of whether Start is still being pressed.

Stop contact (NC): a normally closed contact. Its gap is closed by default (true state, signal passes) until the Stop button is pressed, which opens the gap (false state), breaking the circuit and de-energizing MotorOutput.

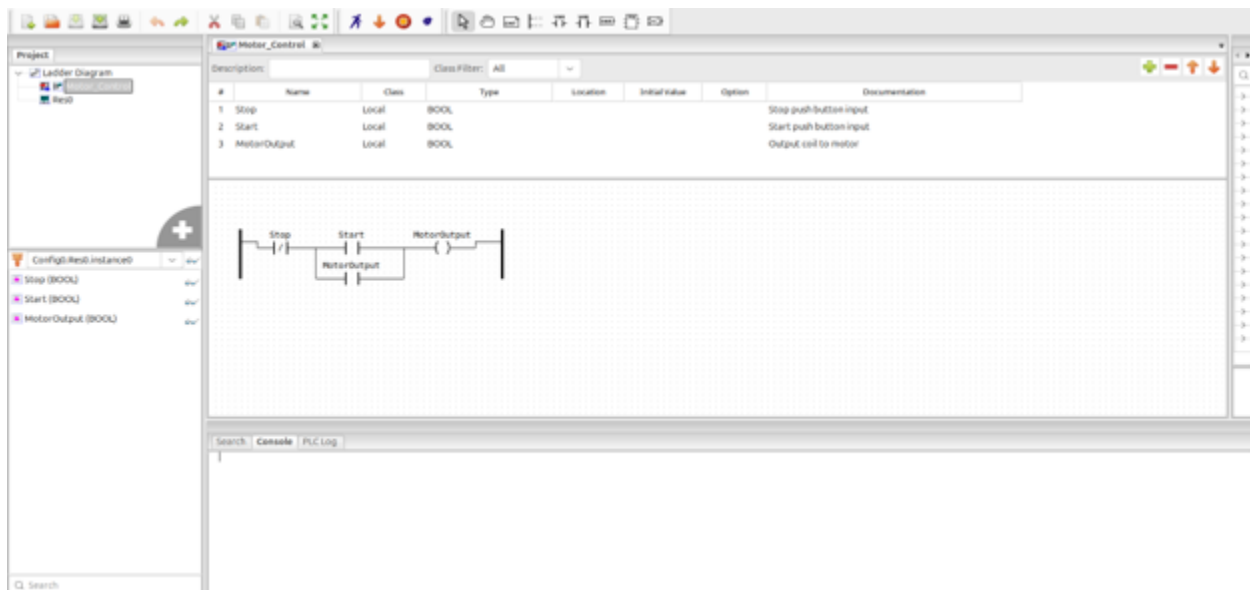
MotorOutput coil: the output element. It energizes when the Stop contact is true (Stop not pressed) AND either the Start contact is true (Start pressed) or the latch contact is already true (motor already running), and sends the resulting signal to the physical motor in the field.

This is the same normally-open/normally-closed distinction covered in the course material: a normally open contact (such as a light switch) is false/open by default and only passes a signal once activated, while a normally closed contact (such as an emergency stop button) is true/closed by default and only breaks the circuit once activated, which is exactly why Stop uses NC logic rather than NO.

Evidence 3: PLC Tags and Boolean Data

Opening the tag/variable list for the Motor_Control program confirms all tags are local to the routine and use the Boolean data type, each with a unique name and short documentation describing its function, as summarized below.

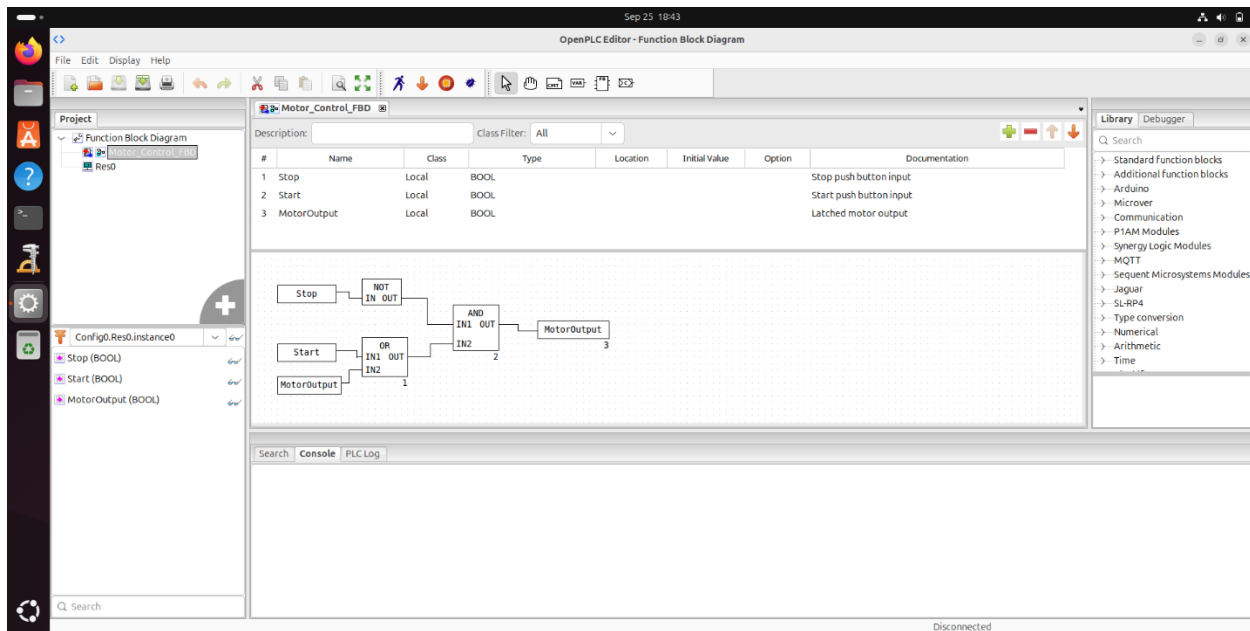
Tag Name	Data Type	Purpose
Start	Boolean (BOOL)	Represents the state of the Start push-button input; TRUE while the button is actively pressed, FALSE otherwise.
Stop	Boolean (BOOL)	Represents the state of the Stop push-button input; TRUE by default (button not pressed, normally closed), FALSE while the button is actively pressed.
MotorOutput	Boolean (BOOL)	Represents both the coil's energized state and, via its own latch contact, the motor's current running/stopped status; TRUE while the motor is commanded on.



Evidence 3 screenshot: OpenPLC Editor tag/variable list for Motor_Control, showing tag names and Boolean data type]

Evidence 4: Function Block Diagram Implementation

Returning to File > Open, the Function Block Diagram project was opened, Res0 expanded, and instance0 (Motor_Control_FBD) opened for inspection.



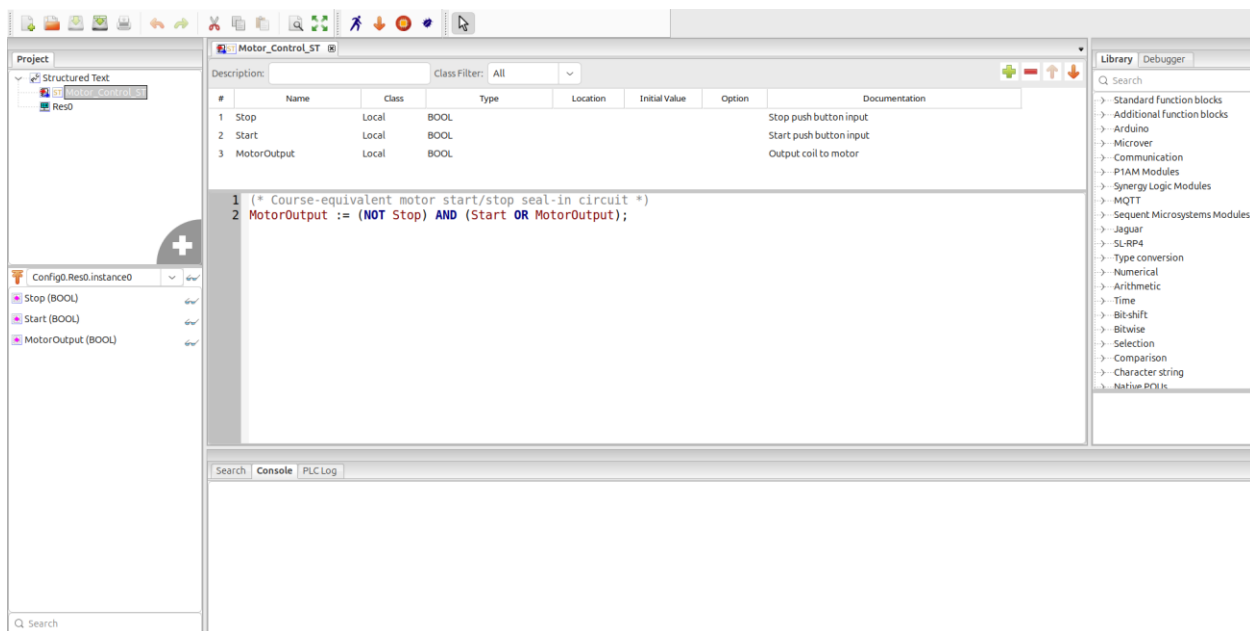
Evidence 4 screenshot: OpenPLC Editor with Motor_Control_FBD fully expanded, program name visible

Function Block Diagram represents the same motor-control requirement using pre-built function blocks connected via pins in a defined execution order, rather than contacts and coils on a rung. In this program, a latching block performs the equivalent role of the Ladder Diagram's Start/latch-contact combination: it holds the motor output signal true once set, until the Stop condition is received, at which point it resets. The underlying control objective, behavior, and tag meanings are identical to the Ladder Diagram version; only the visual/structural representation of the logic differs.

Evidence 5: Structured Text Implementation

The Structured Text project was opened from Downloads/examples/PLC Programming Language Examples/Structured Text, Res0 expanded, and instance0 (MotorControlST) opened for inspection.

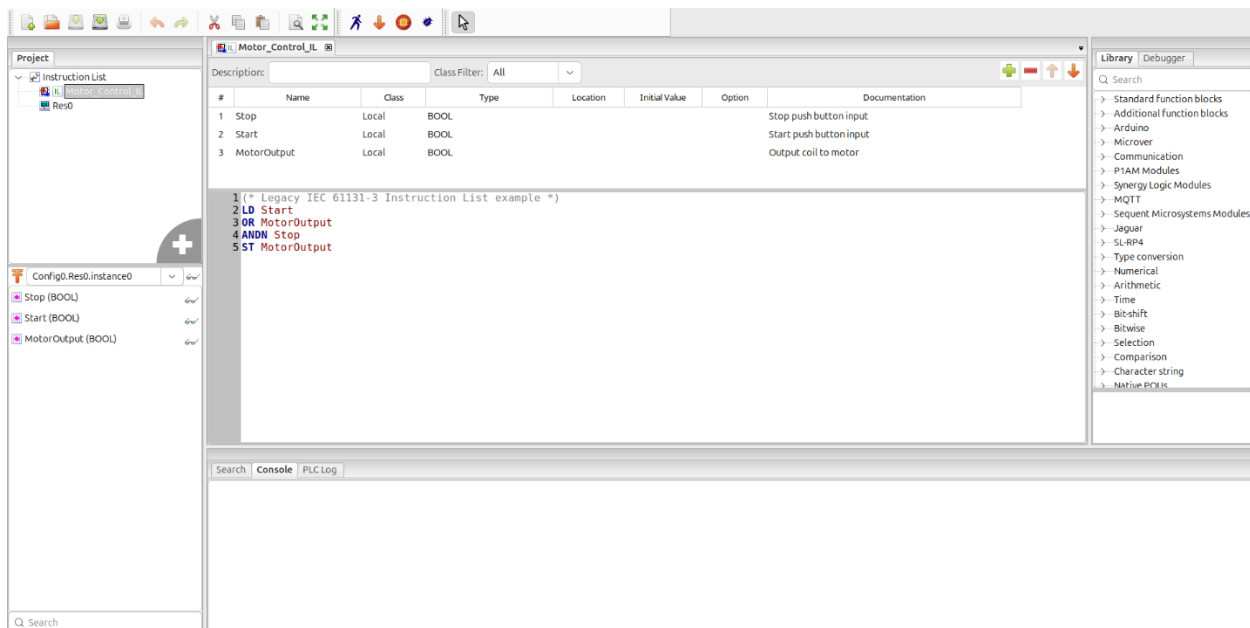
Structured Text is a text-based language developed to mimic the style and syntax of high-level programming languages such as Python, making it more approachable for anyone already familiar with general-purpose programming. The same motor start/stop behavior that requires a full rung of contacts and a coil in Ladder Diagram, or a function block with wired pins in FBD, is expressed here in a single line of conditional logic (in effect, `MotorOutput := (Start OR MotorOutput) AND NOT Stop`, or the equivalent syntax used in the actual program). This demonstrates Structured Text's core strength: performing the same function with substantially less code than the graphical languages.



Evidence 5 screenshot: OpenPLC Editor with MotorControlST fully expanded.

Evidence 6: Instruction List Implementation

The Instruction List project was opened from the Downloads/PLC Programming Language Examples directory, Res0 expanded, and instance0 (MotorControl) opened for inspection.

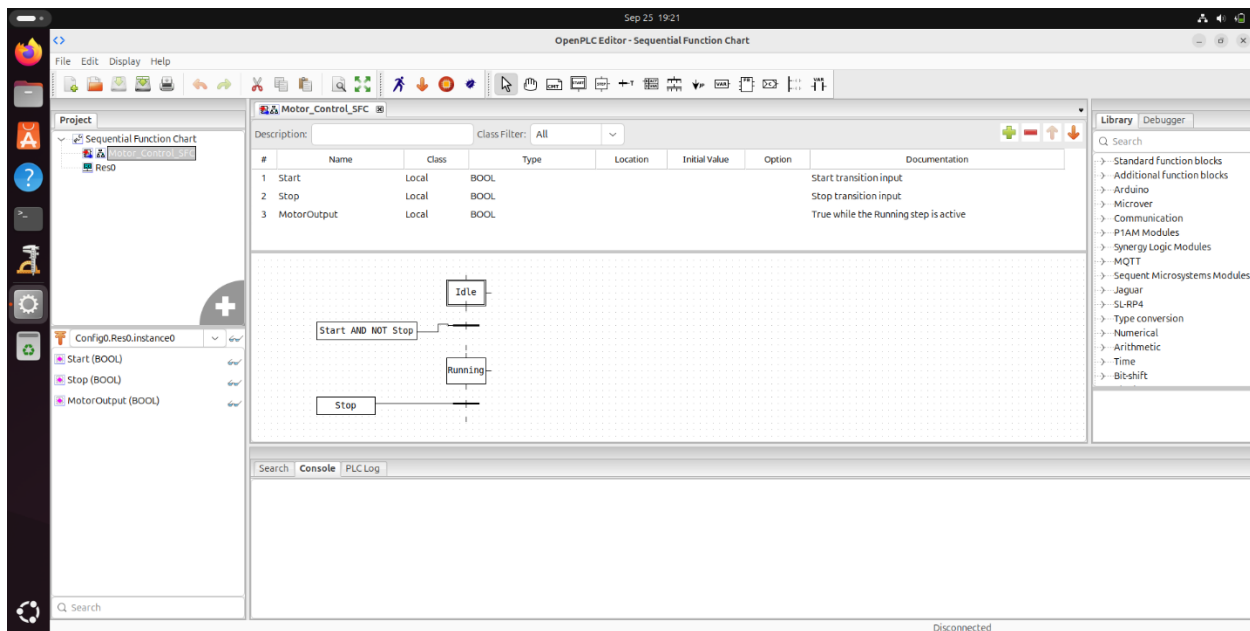
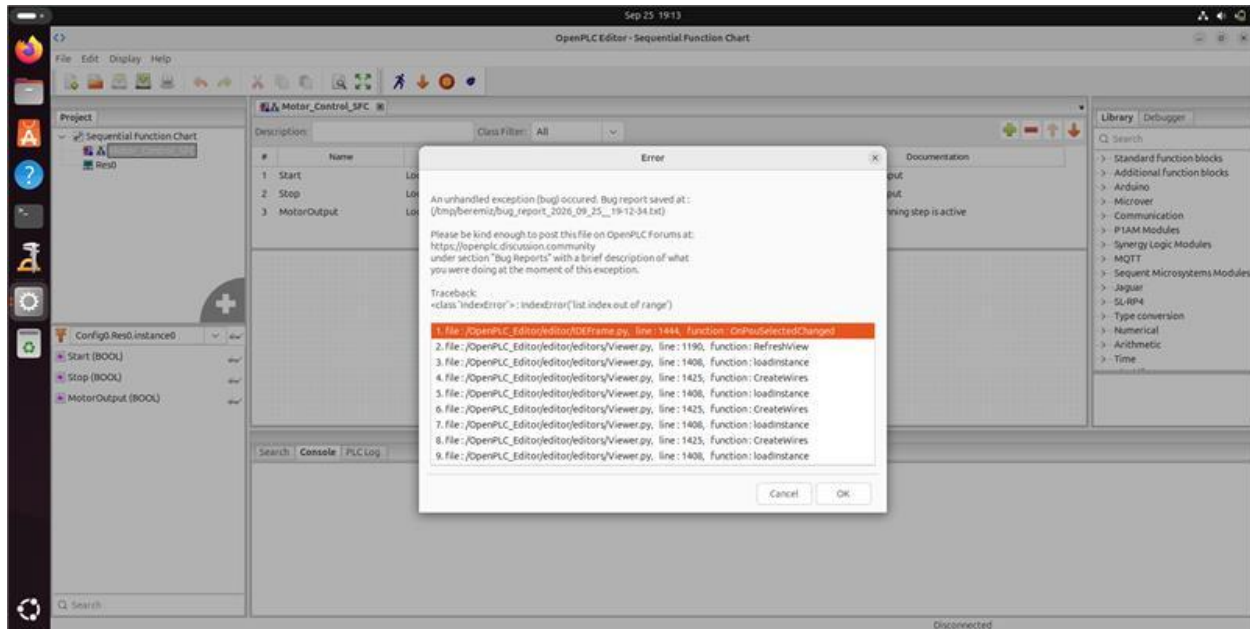


Evidence 6 screenshot: OpenPLC Editor with the Instruction List MotorControl program fully expanded

Instruction List is a low-level, text-based language comparable to assembly language: each instruction is a single operator (such as a load or logical AND/OR operation) placed on its own line, executed strictly in sequence, without the visual rail/rung structure of Ladder Diagram or the block/pin structure of FBD. The main visual difference from Structured Text is density and abstraction level: where Structured Text expresses the motor logic in one readable conditional line, Instruction List breaks the identical logic into several separate, low-level operator instructions.

Evidence 7: Sequential Function Chart Implementation

The Sequential Function Chart project was opened from the Downloads/PLC Programming Language Examples directory, Res0 expanded, and instance0 (MotorControl_SFC) opened for inspection.



Evidence 7 screenshot: OpenPLC Editor with MotorControl_SFC fully expanded

Sequential Function Chart is a graphical language that expresses a process as a sequence of steps and transitions, similar in spirit to a flowchart, rather than as a single evaluated rung, block diagram, or line of text. It can be used alongside Ladder Diagram or Function Block Diagram instructions to step a process through distinct stages, and is commonly used for batch-manufacturing recipes (fill, heat, mix, dispense) where operations must occur in a defined order rather than all being evaluated in parallel on every scan, as the simpler motor start/stop logic is in the other four languages.

Comparative Analysis

Language	Strengths	Limitations	Suitable Industrial Use Case
Ladder Diagram	Visually mirrors physical relay-panel wiring, immediately intuitive to electricians/technicians already trained on relay logic; the most widely used PLC language in industry today.	Can become visually cluttered for complex mathematical or conditional logic compared to text-based languages; less compact than Structured Text for the same function.	General discrete control such as motor start/stop, interlocks, and simple sequencing on the plant floor, exactly the Motor_Control example in this lab.
Function Block Diagram	Reusable, pre-built function blocks (e.g., latches, timers, counters) reduce repeated logic-building; execution order via pin connections is explicit and traceable.	Requires understanding each block's internal behavior rather than reading contacts/coils directly; can be less immediately familiar to relay-trained technicians than Ladder Diagram.	Control systems built from standardized, reusable logic modules, such as PID control loops or repeated latching/timing patterns across many similar machines.
Structured Text	Compact, high-level, text-based syntax similar to Python; well suited to complex calculations, nested conditions, and loops with far less code than graphical languages.	Less visually intuitive for technicians without general programming background; harder to trace physical wiring intuition directly from the code.	Complex data processing, recipe calculations, or conditional logic-heavy applications, such as batch-process calculations or advanced alarming logic.
Instruction List	Very low-level and close to the PLC's native execution, useful for fine-grained control and understanding exactly what the processor executes.	Least common language in industry today; dense, hard to read at a glance, and easy to introduce subtle errors in due to its assembly-like structure.	Legacy systems or very simple, performance-critical routines where a lightweight, low-level instruction set is preferred over graphical overhead.
Sequential Function Chart	Naturally expresses multi-step processes with defined order (steps and transitions), easy to visualize as a process flow rather than parallel logic evaluation.	Not suited to simple, single-condition logic like a basic motor start/stop, where it adds unnecessary structure compared to one ladder rung.	Batch manufacturing recipes and other staged industrial processes (fill → heat → mix → dispense) where steps must execute in a defined sequence.

How the Same Requirement Is Preserved Across Representations

Across all five implementations examined in this lab, the underlying control objective is identical: the motor output must energize when Start is pressed and remain energized until Stop is pressed and must never restart on its own once stopped. What changes between languages is only the representation, graphical contacts and a coil in Ladder Diagram, a connected latch block in FBD, a single conditional expression in Structured Text, a sequence of low-level operators in Instruction List, or a defined step/transition sequence in Sequential Function Chart, not the physical behavior being described. The Start, Stop, and MotorOutput tags retain the same Boolean meaning in every version, which is precisely what allows the same requirement to be re-expressed in whichever language best suits the task (intuitive relay-style logic, reusable function blocks, compact calculation, low-level control, or staged sequencing) without changing what the motor actually does in the field.

Knowledge Check

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Ladder Diagram visually mirrors the physical relay-panel wiring these technicians already read and troubleshoot: the PowerRails represent the same power-supply wires, the rungs represent the same inter-component wiring, and the NO/NC contact and coil symbols are based directly on the mechanical relay components used in real panels. A technician can effectively read the ladder diagram the same way they would read the physical relay circuit it replaces, rather than needing to learn an unfamiliar abstraction.

2. What is one advantage of Structured Text for complex calculations or conditions?

Structured Text's high-level, text-based syntax (similar to Python) can express complex mathematical formulas and nested conditional logic far more compactly than building the equivalent from graphical contacts, coils, or function blocks; this lab's own example needed only one line of Structured Text to reproduce logic that takes a full rung in Ladder Diagram or a wired block in FBD.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

When the process involves multiple distinct stages that must occur in a defined order, such as a batch-manufacturing recipe (fill, then heat, then mix, then dispense), rather than a single condition being evaluated in parallel every scan. A single ladder rung evaluates its logic continuously and in parallel; SFC's steps and transitions naturally express “do not begin step 2 until step 1's condition is satisfied,” which a basic rung does not express as directly.

4. Why must the meaning of tags remain consistent when the representation/language changes?

Because the physical control objective (what Start, Stop, and MotorOutput actually mean to the real motor in the field) does not change just because the programming language used to express it does. If a tag's meaning shifted between representations, the “same” program could actually command different physical behavior depending on which language happened to be open, undermining any confidence that translating or comparing logic across languages preserves the intended real-world outcome, which matters even more where a misinterpreted tag could result in an unsafe or unintended physical action.

Student Declaration

- I confirm that this submission is my own work and that all sources, screenshots, simulations, and calculations used have been honestly presented.
- I understand that unsafe electrical practice, copied work, or fabricated screenshots may attract disciplinary action.

Student Name	Nathaniel Temitope Ogunjobi
Registration Number	C11/26/ACIS/17372
Signature	
Date	22 September 2026